High Performance Computing (HPC)

Assignment 08 – Parallel Interpolation with Particle Mover using MPI

Instructor: Dr. Bhaskar Chaudhury
Teaching Assistant: Ayushi Sharma, Libin Varghese, Kaushik Prajapati

April 8, 2026

General Instructions

- This assignment focuses on **parallelizing** the given interpolation algorithm.
- You must begin with a **correct serial baseline** and then develop a parallel implementation.
- You have to use **OpenMP and MPI** for parallelization.
- The evaluation will be based on **correctness, scalability, performance, and analysis**.
- You are encouraged to experiment with **data structures, memory layout, and parallelization strategies**.
- Submission will be done through a separate Google Form (link will be shared).

---

1 Recap from the Previous Assignment

In Assignment 03, we introduced the scatter point-to-mesh interpolation problem, where a set of points was randomly initialized inside a 1×1 domain and their values were interpolated onto a structured $N_x \times N_y$ grid. The objective was to study how the execution time varies with different grid sizes and point counts. In the previous assignment, the scatter interpolation process (point $\rightarrow$ mesh), along with the mover and reverse interpolation (mesh $\rightarrow$ point), was parallelised using OpenMP. In the current assignment, the focus is on implementing the same overall workflow using MPI + OpenMP.

2 Problem Description

Given a set of scattered points in a 2D domain with known function values:

$$S = \{(x_i, y_i, f_i) \mid i = 1, 2, \ldots, N\}$$

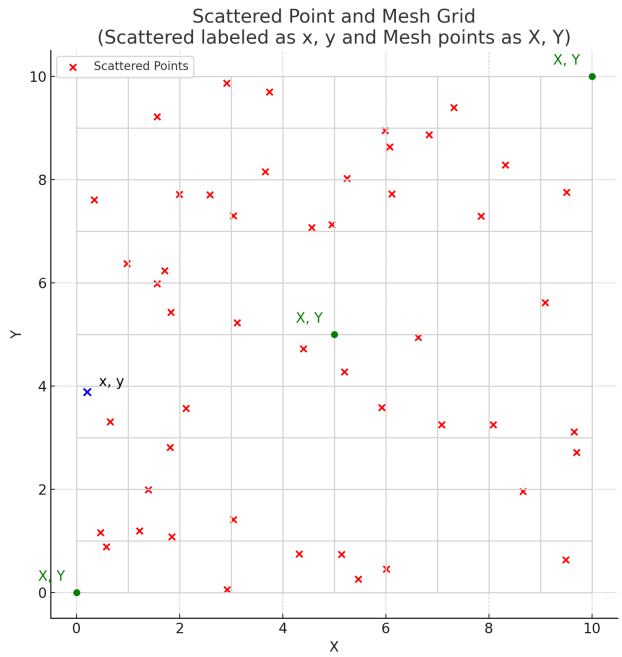

Figure 1: Figure of the domain with scattered points

where (x_i, y_i) are spatial coordinates and f_i are known function values. For this assignment, assume $f_i = 1$. The task is to interpolate these values onto a structured mesh grid of size $M \times M$.

The structured mesh consists of regularly spaced grid points:

$$G = \{(X_i, Y_j) \mid X_i = i\Delta x, \quad Y_j = j\Delta y, \quad i, j = 0, 1, \ldots, M-1\}$$

where Δx and Δy define the uniform grid spacing and $0 \leq x_i \leq X_{max}$ and $0 \leq y_i \leq Y_{max}$. See Fig.1 for reference.

$$\Delta x = \frac{X_{max}}{M}, \quad \Delta y = \frac{Y_{max}}{M}$$

All scattered points lie within a domain of size 1×1.
Typical use of this scheme is mentioned here.

1. **Computer Graphics and 3D Modeling**:
   - Used to generate smooth surfaces from scattered 3D points, such as those obtained from 3D scanning (e.g., LIDAR or photogrammetry). For example, turning a point cloud of a scanned object into a mesh for rendering or animation.
2. **Scientific Visualization**:
   - In fields like meteorology or oceanography, scattered data points (e.g., temperature or pressure readings from sensors) are interpolated onto a mesh to visualize continuous fields over a region.
3. **Finite Element Analysis (FEA)**:

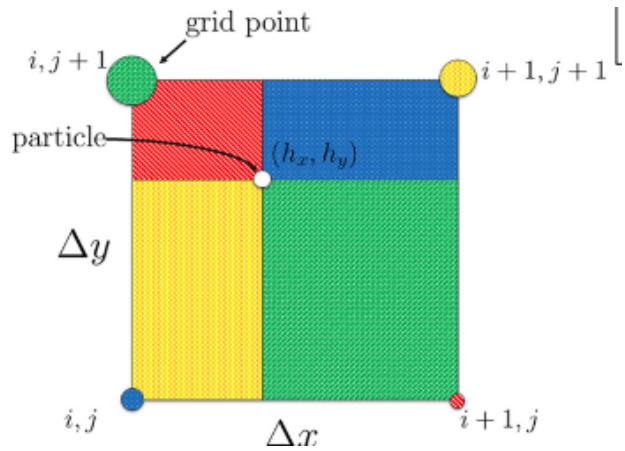

Figure 2: Linear interpolation illustration. Here h_x,h_y correspond to x_i,y_i

- Engineers use this in simulations (e.g., structural analysis, fluid dynamics) to map scattered data (like material properties or experimental measurements) onto a structured mesh for numerical solving.

4. **Medical Imaging**:
   - Reconstructing surfaces or volumes from scattered data points, such as creating a 3D model of an organ from MRI or CT scan points.
5. **Machine Learning and Data Science**:
   - When dealing with sparse or unevenly sampled data, this scheme can help create a continuous representation for tasks like regression, visualization, or simulation.

3 Linear Interpolation

Linear interpolation is a simple **bilinear interpolation** method in which each scattered point distributes its value to the **four nearest grid points**, proportionally to its relative position inside the grid cell. The procedure consists of:

1. Identify the four nearest grid points (X_i, Y_j) surrounding a scattered point (x_i, y_i).
2. Compute the weights based on the relative distances of (x_i, y_i) to these grid points.
3. Distribute the value f_i proportionally to these grid points.

For a scattered point (x_i, y_i) located in a grid cell with corners at: (X_i, Y_j), (X_{i+1}, Y_j), (X_i, Y_{j+1}), (X_{i+1}, Y_{j+1}). The weight factors are computed as:

$$w_{i,j} = (\Delta x - l_x) \times (\Delta y - l_y)$$

$$w_{i+1,j} = l_y \times (\Delta x - l_x)$$

$$w_{i,j+1} = l_x \times (\Delta y - l_y)$$

$$w_{i+1,j+1} = l_x \times l_y$$

where:

$$l_x = x_i - X_i, \quad l_y = y_j - Y_j$$

Since the values of the grid point are not necessary to be stored explicitly, the indices can be calculated on the fly:

$$X_i = \left\lfloor \frac{x_i}{\Delta x} \right\rfloor \times \Delta x, \quad Y_j = \left\lfloor \frac{y_i}{\Delta y} \right\rfloor \times \Delta y$$

The interpolated function values at the structured grid points are accumulated as:

$$F(X_i, Y_j) += w_{i,j} \times f_i$$
$$F(X_{i+1}, Y_j) += w_{i+1,j} \times f_i$$
$$F(X_i, Y_{j+1}) += w_{i,j+1} \times f_i$$
$$F(X_{i+1}, Y_{j+1}) += w_{i+1,j+1} \times f_i$$

For example, the value to be stored on grid point (i, j) depends on the value of weight $w_{i,j}$, which is calculated using the area of the blue region as shown in Fig. 2.

In a serial implementation, interpolation is straightforward and requires two data structures: one for storing scattered points and another for storing interpolated values on a structured mesh. However, in a parallel implementation, special care must be taken to eliminate dependencies. This is because multiple scattered points may fall within the same or adjacent cells, leading to shared grid points and potential race conditions. To improve performance, these dependencies must be carefully managed. **This can be achieved by modifying the data structures or optimizing the parallel implementation to ensure efficient computation. Your final scores will be evaluated based on the leaderboard, and your results will be shared in Google Classroom. Your end goal is to increase the performance of the existing code for a large number of threads.**

4 Particle Update using Reverse Interpolation (Mover)

In addition to mapping particle values to the grid, we now introduce a **reverse interpolation step**, commonly referred to as a **mover**. In this step, the grid values influence the motion of particles.
The idea is as follows:

- First, interpolation is performed from particles to the grid.
- The grid values are then normalized to a fixed range.
- Using the same interpolation weights, values are interpolated **back from the grid to particles**.
- These interpolated values are used to update particle positions.

For a particle located at (x_i, y_i), the field value is computed using the same four neighboring grid points:

$$F_i = w_{i,j} \cdot F(X_i, Y_j) + w_{i+1,j} \cdot F(X_{i+1}, Y_j) + w_{i,j+1} \cdot F(X_i, Y_{j+1}) + w_{i+1,j+1} \cdot F(X_{i+1}, Y_{j+1})$$

The particle position is then updated as:

$$x_i^{new} = x_i + F_i \cdot \Delta x$$
$$y_i^{new} = y_i + F_i \cdot \Delta y$$

If a particle moves outside the domain, it is marked as inactive and excluded from further computations.
This step introduces additional computational complexity and must also be parallelized efficiently.

5 Implementation Steps

1. **Read input data**: Load scattered points (x_i, y_i, f_i) from *input.bin*. The data in this file is in binary format (to save the space). The domain size is 1 unit $\times$ 1 unit. All particles and grid points should lie within this domain.

2. **Initialize structured mesh**: Define the grid G and set all function values $F(X_i, Y_j)$ to zero.

3. **Perform interpolation and particle update loop**:

   - Loop through each scattered point and perform interpolation onto the grid.
   - Compute weights and distribute values to neighboring grid points.
   - Normalize the grid values to the range $[-1, 1]$.
   - Perform reverse interpolation (mover):
     - Interpolate grid values back to each particle.
     - Update particle positions using the interpolated field.
     - Mark particles outside the domain as inactive.
   - Denormalize the grid values back to original range.
   - This entire pipeline must be repeated for each iteration.
   - Think carefully about memory access patterns and data dependencies.
   - First, write a correct serial implementation and validate results.
   - You are free to use any data structure that improves performance.

4. The interpolation + mover pipeline will be executed for multiple iterations using the same set of particles. Particle positions evolve over time based on the computed grid field.

5. **Output the interpolated grid**: Write the structured mesh values to an output file.

6. To check accuracy, run the code using `test_input.bin` and compare with `test_output.txt`. **Performance improvement is important but not at the cost of correctness.**

7. Use *input_fileMaker.c* to generate input data with the following configurations:

(a) N_x=250, N_y=100, points=0.9 million, Maxiter=10
(b) N_x=250, N_y=100, points=5 million, Maxiter=10
(c) N_x=500, N_y=200, points=3.6 million, Maxiter=10
(d) N_x=500, N_y=200, points=20 million, Maxiter=10
(e) N_x=1000, N_y=400, points=14 million, Maxiter=10

6 Parallelization Objective

The goal of this assignment is to transform the given serial implementation into an efficient **parallel version of the complete pipeline**, which includes both interpolation and particle update (mover) steps.

In the interpolation phase, scattered particles contribute to nearby grid points, which introduces **race conditions** as multiple particles may update the same grid locations simultaneously. In contrast, the mover phase involves interpolating grid values back to particles, which is primarily a **read-heavy operation**, but must be executed efficiently for a large number of particles.

Additionally, since particle positions are updated after each iteration, the computation introduces **data dependency across iterations**. Over time, some particles may move outside the domain and become inactive, which can further lead to **load imbalance** across threads.

To achieve good performance, you must carefully design:

- Efficient handling of **race conditions** in the interpolation step
- **Memory access patterns** for both grid and particle data
- Proper **load balancing** as particle distribution evolves
- Techniques to **minimize synchronization overhead**

You are expected to explore different parallelization strategies such as:

- Domain decomposition
- Particle decomposition
- Use of atomic operations vs privatization
- Reduction-based approaches

Your implementation must efficiently parallelize **both the interpolation and mover phases** while maintaining correctness.

In this assignment, you are expected to make a hybrid (MPI+OpenMP) code. The MPI parallelization needs to be added to the existing code to include communication across nodes. This MPI layer will enable you to use all the cores available on the HPC.

7 Analysis and Questions

1. Write the pseudocode of your parallel implementation in the report along with an illustrative diagram of your implementation.
2. Explain your parallel implementation approach and how you handled race conditions in your code.
3. Plot graphs for each of the five input configurations for speedup vs cores and execution time vs cores (increment cores starting from 2 to 64 by a factor of 2). Ensure no one else is using the machine while running the code to get good performance. Explain the difference between a login node and a compute node. You are expected to run your code on the compute nodes (gics1-4) you should make sure code is run in all 4 nodes.
4. Compute the parallel efficiency and comment on the scalability of your implementation.
5. Compare the performance of your parallel code with the serial version and report the maximum speedup achieved.
6. Try to explain the speedup achieved and all the observations using your current HPC knowledge.
7. If given unlimited HPC resources, how would you further optimize the implementation?
8. Discuss any load imbalance or performance bottlenecks observed in your implementation.
9. Suggest at least one alternative approach (data structure or parallelization method) to improve performance.
10. You are required to develop alternative data structures or OpenMP-MPI-based approaches to improve the code's performance. Anyone who gets good results may receive bonus marks for this assignment. Leaderboard will be based on performance and efficiency.
11. Analyze the performance of interpolation and mover phases separately. Which phase is the bottleneck and why?

Note: Ensure that the implementation correctly interpolates values. The accuracy of your code will be tested against a hidden dataset. Incorrect implementations will not be evaluated for performance.

8 Submission Guidelines

- Submit a PDF report including:
  - Explanation of your implementation approach
  - Performance analysis
  - Answers to analysis questions
- Submit source code in compressed format.
- Submission will be done through a separate Google Form (link will be shared).

9 Code Repository and Remarks

The base implementation and supporting files are available in the shared GitHub repository: Click here
With so much information and resources at your disposal, sincere effort is expected. **All the best.**